

Spring Framework — Bean Lifecycle

Topic: Spring Bean Lifecycle

Focus: How Spring creates, initializes, uses, and destroys Beans

Style: Sequence-wise + interview-oriented

1. What is Bean Lifecycle?

A Spring Bean goes through multiple stages from **creation to destruction**.

The IoC Container is responsible for managing the lifecycle of Spring-managed Beans.

Basic Flow

```
IoC Container Starts
    ↓
Read Configuration
    ↓
Scan Components
    ↓
Create Bean Definitions
    ↓
Instantiate Objects
    ↓
Inject Dependencies
    ↓
Aware Interfaces / Callbacks
    ↓
Initialization Callbacks
    ↓
Bean Ready
    ↓
Bean Used
    ↓
Destruction Callbacks
    ↓
Bean Destroyed
```

2. Bean Lifecycle — Complete Sequence

For a typical **Singleton + Eager Bean**, the sequence discussed in the chapter is:

1. IoC Container starts
2. Configuration is read
3. Components are scanned
4. Bean Definitions are created
5. Bean objects are instantiated
6. Dependencies are injected/resolved
7. Aware interfaces are called
8. Initialization callbacks are called
9. Bean becomes ready to use
10. Bean is used
11. Destruction callbacks are called
12. Bean is destroyed when the container is closed

Important: Some stages can effectively merge depending on how dependency injection is performed.

3. Step 1 — IoC Container Starts

The first step is starting the Spring IoC Container.

Example:

```
ApplicationContext context =  
    new AnnotationConfigApplicationContext(AppConfig.class);
```

The `ApplicationContext` is used to interact with the Spring IoC Container.

4. Step 2 — Configuration is Read

Spring needs configuration information to know:

- Which components to manage
- Which packages to scan
- Which `@Bean` methods exist
- Bean-related metadata

Example:

```
@Configuration
@ComponentScan
public class AppConfig {
}
```

Spring reads the metadata of `AppConfig`.

5. Reflection and Metadata

Spring uses Java reflection to inspect classes and their metadata.

Reflection can provide information such as:

```
Class information
Annotations
Constructors
Methods
Fields
Modifiers
```

For example:

```
AppConfig.class
```

provides the class metadata that Spring can inspect.

6. @ComponentScan

If:

```
@ComponentScan
```

is used without specifying a package, Spring scans the package associated with the configuration class and its relevant subpackages.

Spring looks for classes such as:

```
@Component
@Service
@Repository
@Controller
```

and other component stereotypes.

7. Step 3 — Bean Definitions are Created

Before creating actual Bean objects, Spring creates **Bean Definitions**.

What is a Bean Definition?

A Bean Definition contains metadata/instructions about how Spring should manage a Bean.

It can contain information such as:

```
Bean Name
Bean Class
Scope
Lazy/Eager information
Dependencies
Initialization information
Destruction information
```

Example:

```
Bean Definition
↓
Bean Name      → orderService
Bean Class     → OrderService
Scope          → Singleton
Lazy           → false
Dependencies   → PaymentService
```

8. Why Create Bean Definitions First?

Spring does not simply start creating objects randomly.

It first collects Bean Definitions so that it knows:

```
Which Beans exist?  
Which class creates them?  
What is their scope?  
Are they lazy?  
What dependencies do they have?
```

This allows Spring to determine how Beans should be created and managed.

9. All Bean Definitions are Read First

Spring first collects the definitions of the relevant Beans.

For example:

```
OrderService    → Bean Definition  
PaymentService → Bean Definition  
CardService     → Bean Definition
```

Only after this metadata is available does Spring proceed with Bean creation.

10. Step 4 — Instantiate Bean Objects

Now Spring creates the actual Java objects.

This is called:

```
Instantiation
```

Example conceptually:

```
new PaymentService();
```

and:

```
new OrderService(...);
```

The actual objects become Spring-managed Bean instances.

11. Spring is Dependency-Aware

Spring does not necessarily create Beans in a random order.

Suppose:

```
OrderService
  ↓
PaymentService
```

`OrderService` requires `PaymentService`.

Spring understands this dependency.

Therefore:

```
Create OrderService
  ↓
Requires PaymentService
  ↓
Create PaymentService first
  ↓
PaymentService ready
  ↓
Create OrderService
```

12. Constructor Injection and Object Creation

Suppose:

```
@Component
class OrderService {

    private final PaymentService paymentService;
```

```
    OrderService(PaymentService paymentService) {  
        this.paymentService = paymentService;  
    }  
}
```

With constructor injection, the dependency is required **during construction**.

Therefore:

```
Create OrderService  
    ↓  
Constructor requires PaymentService  
    ↓  
PaymentService must be available  
    ↓  
PaymentService injected into constructor  
    ↓  
OrderService object created
```

So with constructor injection:

Instantiation + Dependency Injection

are closely connected.

13. Field / Setter Injection

If dependency injection happens through a field or setter:

```
@Autowired  
private PaymentService paymentService;
```

or:

```
@Autowired  
public void setPaymentService(PaymentService paymentService) {  
    this.paymentService = paymentService;  
}
```

the object can first be instantiated and the dependency can be injected afterward.

Conceptually:

```
Object created
  ↓
Dependency injected
```

Therefore, the lifecycle stages are more clearly separated.

14. Step 5 — Dependencies are Injected

Once required objects are available, Spring resolves and injects dependencies.

Example:

```
OrderService
  ↓
PaymentService
```

After injection:

```
OrderService → PaymentService Object
```

The Bean now has its required dependencies.

15. Important: Aware Interfaces

After dependency injection, Spring can call certain **Aware interfaces** implemented by the Bean.

Aware interfaces allow a Bean to receive information from the Spring container.

Examples discussed:

```
BeanNameAware
ApplicationContextAware
```

16. BeanNameAware

If a Bean implements:

BeanNameAware

Spring calls:

setBeanName(String name)

Example:

```
@Component("userBean")
public class UserService implements BeanNameAware {

    @Override
    public void setBeanName(String name) {
        System.out.println(name);
    }
}
```

Spring supplies the Bean's name.

Output:

userBean

17. ApplicationContextAware

A Bean can implement:

ApplicationContextAware

Spring then calls:

setApplicationContext(ApplicationContext applicationContext)

This gives the Bean access to the ApplicationContext.

Conceptually:

```
Bean
↓
Spring calls setApplicationContext()
↓
ApplicationContext supplied
```

18. Aware Methods are Callback Methods

You normally do **not** call these methods yourself.

Spring calls them as part of Bean lifecycle processing.

Therefore:

```
Spring → calls method
```

is a:

```
Callback
```

Important

If you manually call:

```
setBeanName("abc");
```

you are only calling a normal Java method yourself.

It does **not** change the actual Bean name stored in the Spring container.

The actual Bean name comes from Spring's Bean definition.

19. Why Use Aware Interfaces?

The chapter emphasizes that Aware interfaces have relatively **specific use cases**.

They can be useful when a Bean needs framework-level information, such as:

```
Bean name
ApplicationContext
```

One example discussed is logging/debugging.

For example:

```
[OrderService] Error occurred...
```

Knowing the Bean name or container can help identify where a log originated.

But they are generally **not required for normal business logic**.

20. Step 6 — Initialization Callbacks

After:

```
Object creation
    ↓
Dependency Injection
    ↓
Aware callbacks
```

Spring performs the **initialization callback phase**.

Purpose:

Perform initialization work after the Bean has been created and its dependencies are available, but before the Bean is used.

21. Why Initialization Callbacks?

Suppose a Bean needs to:

```
Initialize a Map
Load initial data
Clear a cache
Invalidate a cache
Prepare a resource
Load an expensive resource
Perform startup configuration
```

These operations can be performed in the initialization phase.

Example:

```
Bean Created
  ↓
Dependencies Ready
  ↓
Initialization
  ↓
Bean Ready
```

22. Three Initialization Callback Approaches

The chapter discusses three approaches:

1. InitializingBean
2. init-method
3. @PostConstruct

23. Method 1 — InitializingBean

`InitializingBean` is an interface.

It provides:

```
afterPropertiesSet()
```

Example:

```
@Component
public class CardService implements InitializingBean {

    @Override
    public void afterPropertiesSet() {
        // initialization logic
    }
}
```

Spring calls:

```
afterPropertiesSet()
```

during Bean initialization.

Important

This approach is older and is generally less preferred than annotation-based initialization.

24. Method 2 — `init-method`

An initialization method can be specified for a Bean.

Example concept:

```
@Bean(initMethod = "start")
public CardService cardService() {
    return new CardService();
}
```

Bean class:

```
public class CardService {

    public void start() {
```

```
        // initialization logic
    }
}
```

Spring calls:

```
start()
```

during initialization.

Advantage

You don't need to implement:

```
InitializingBean
```

You can define your own method.

25. Method 3 — @PostConstruct

This is the commonly used annotation-based approach discussed in the chapter.

Example:

```
@Component
public class CardService {

    @PostConstruct
    public void start() {
        // initialization logic
    }
}
```

Spring automatically calls the annotated method during Bean initialization.

No interface implementation is required.

No method override is required.

26. @PostConstruct — Simple Flow

```
Constructor
  ↓
Dependency Injection
  ↓
Aware callbacks
  ↓
@PostConstruct
  ↓
Bean Ready
```

The important point is:

```
@PostConstruct
→ called after dependency injection
→ before normal Bean usage
```

27. Why Not Put Everything in Constructor?

A constructor executes when the object is being created.

At that stage, with field/setter injection:

```
Constructor
  ↓
Dependencies may not yet be injected
```

But `@PostConstruct` runs later:

```
Constructor
  ↓
Dependency Injection
  ↓
Aware callbacks
  ↓
@PostConstruct
```

Therefore, initialization logic that requires injected dependencies can be placed in the initialization phase.

28. Constructor Should Be Lightweight

The chapter emphasizes avoiding heavy work in the constructor.

Prefer:

```
Constructor  
→ Basic object creation
```

and:

```
@PostConstruct  
→ Initialization work
```

For example:

```
Cache initialization  
Resource preparation  
Data loading  
Map initialization
```

can be performed during initialization.

29. Why `@PostConstruct` is Useful

Suppose:

```
@Autowired  
private SomeService service;
```

If initialization requires `service`, doing it inside the constructor is problematic because field injection has not happened yet.

`@PostConstruct` runs after dependency injection.

Therefore:

```
Dependency available
  ↓
@PostConstruct
  ↓
Use dependency for initialization
```

30. Bean is Ready

After initialization callbacks finish:

```
Bean is Ready
```

Now the application can use the Bean normally.

Example:

```
CardService card =
    context.getBean(CardService.class);

card.getValue();
```

31. Step 7 — Bean is Used

Once initialization is complete:

```
Bean Ready
  ↓
Business Methods
  ↓
Bean Used
```

Example:

```
card.addToCart();
card.getValue();
```

The Bean remains available according to its scope.

32. Destruction Callbacks

After the Bean has finished its work, Spring may need to destroy it.

Before destruction, Spring provides another callback phase:

Destruction Callback

Purpose:

Perform cleanup before a Spring-managed Bean is destroyed.

33. What Can Be Done in Destruction Callback?

Typical cleanup operations discussed include:

Clear Map
Clear Cache
Invalidate Cache
Close Expensive Resources
Release Resources
Cleanup temporary state

Conceptually:

Initialization
→ Acquire / prepare resources

Destruction
→ Release / close resources

34. Three Destruction Callback Approaches

The chapter gives three corresponding approaches:

1. DisposableBean
2. destroy-method
3. @PreDestroy

35. Method 1 — DisposableBean

`DisposableBean` is an interface.

It provides:

```
destroy()
```

Example:

```
@Component
public class CardService implements DisposableBean {

    @Override
    public void destroy() {
        // cleanup logic
    }
}
```

Spring calls:

```
destroy()
```

during Bean destruction.

36. Method 2 — destroy-method

Similar to `init-method`, a custom destruction method can be specified.

Example:

```
@Bean(
    initMethod = "start",
```

```
        destroyMethod = "stop"
    )
    public CardService cardService() {
        return new CardService();
    }
}
```

Bean:

```
public void start() {
    // initialization
}

public void stop() {
    // cleanup
}
```

Spring calls:

```
start()
→ initialization

stop()
→ destruction
```

37. Method 3 — @PreDestroy

The annotation-based destruction callback is:

```
@PreDestroy
```

Example:

```
@Component
public class CardService {

    @PreDestroy
    public void cleanup() {
        // cleanup logic
    }
}
```

```
}  
}
```

Spring calls this method before destroying the Bean.

38. @PostConstruct vs @PreDestroy

Annotation	When Called	Purpose
@PostConstruct	After Bean creation/dependency injection	Initialization
@PreDestroy	Before Bean destruction	Cleanup

Easy Memory Trick

```
@PostConstruct  
→ Bean is constructed  
→ Initialize  
  
@PreDestroy  
→ Bean is about to be destroyed  
→ Cleanup
```

39. Closing the ApplicationContext

To simulate Bean destruction, the chapter closes the context.

`ApplicationContext` itself does not expose `close()` directly.

A configurable context can be used:

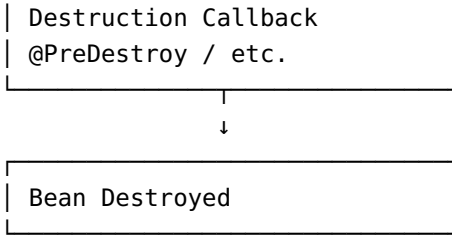
```
ConfigurableApplicationContext context =  
    new AnnotationConfigApplicationContext(AppConfig.class);  
  
context.close();
```

When the context closes, Spring performs destruction processing for applicable managed Beans.

40. Complete Singleton Bean Lifecycle

For the Singleton Bean discussed in the chapter:





41. Initialization Callback — Three Ways

```
InitializingBean
  ↓
afterPropertiesSet()

init-method
  ↓
Custom method

@PostConstruct
  ↓
Annotated method
```

42. Destruction Callback — Three Ways

```
DisposableBean
  ↓
destroy()

destroy-method
  ↓
Custom method

@PreDestroy
  ↓
Annotated method
```

43. Prototype Bean Lifecycle

Prototype scope behaves differently from Singleton.

When:

```
@Scope("prototype")
```

is used, Spring creates a **new Bean instance each time the Bean is requested from the container.**

Example:

```
CardService c1 =  
    context.getBean(CardService.class);  
  
CardService c2 =  
    context.getBean(CardService.class);
```

Conceptually:

```
getBean()  
  ↓  
Object #1  
  
getBean()  
  ↓  
Object #2
```

Therefore:

```
c1 == c2
```

is:

```
false
```

44. Prototype Lifecycle

The creation-side lifecycle still includes the relevant creation and initialization processing:

```
Bean Definition
  ↓
Instantiate
  ↓
Dependency Injection
  ↓
Aware Callbacks
  ↓
Initialization Callbacks
  ↓
Bean Handed to Client
```

But after that:

```
Spring stops managing the prototype instance's full lifecycle
```

45. Important Prototype Rule

For a Prototype Bean:

```
Spring creates it
  ↓
Spring initializes it
  ↓
Spring gives it to the client
  ↓
Spring does NOT manage its destruction
```

Therefore:

```
@PreDestroy
```

is not automatically called by Spring for a prototype Bean when that prototype instance is no longer needed.

46. Why Doesn't Spring Manage Prototype Destruction?

A prototype Bean can produce many objects:

```
getBean()  
→ Object #1  
  
getBean()  
→ Object #2  
  
getBean()  
→ Object #3  
  
getBean()  
→ Object #4
```

If Spring retained every prototype instance until context shutdown, the container could accumulate many objects.

The chapter explains this in terms of avoiding unnecessary memory retention/memory-leak-like behavior.

Therefore, Spring does not track prototype instances for complete destruction management.

47. How is a Prototype Object Eventually Removed?

After the client no longer holds references to the object, normal Java memory management applies.

Conceptually:

```
Prototype Object  
↓  
No references  
↓  
Eligible for Garbage Collection  
↓  
GC eventually removes object
```

Spring does not call its destruction callback as it does for applicable Singleton Beans.

48. `context.close()` and Prototype Beans

Closing the Spring context does **not** mean Spring destroys every Prototype instance it has ever created.

Why?

Prototype instance
→ handed to client
→ no longer managed as a full lifecycle object by Spring

Therefore:

```
context.close()
```

does not provide the same destruction handling for prototype instances.

49. Prototype + Expensive Resources

If a Prototype Bean owns an external/expensive resource, you should not assume Spring will automatically clean that resource up through `@PreDestroy`.

The application/client must handle resource cleanup appropriately.

Examples:

Files
Connections
Streams
External resources

Use proper Java resource-management techniques where applicable.

50. Singleton vs Prototype Lifecycle

Stage	Singleton	Prototype
Bean Definition	Yes	Yes

Stage	Singleton	Prototype
Object Creation	Yes	Yes
Dependency Injection	Yes	Yes
Aware Callbacks	Yes	Yes
Initialization Callback	Yes	Yes
Given to Client	Yes	Yes
Destruction managed by Spring	Yes	No
<code>@PreDestroy</code> automatically handled by Spring	Yes, for applicable managed Singleton Beans	No

51. Circular Dependency Using `@PostConstruct`

The chapter also demonstrates an interesting way of breaking a circular dependency using setter injection + `@PostConstruct`.

Suppose:

```
A → B
B → A
```

Using constructor injection:

```
class A {
    A(B b) { }
}
```

```
class B {
    B(A a) { }
}
```

This creates:

```
Create A
  ↓
Need B
  ↓
```

```
Create B
  ↓
Need A
  ↓
Circular Dependency
```

52. Breaking the Cycle

Instead of constructor injection in both directions, one side can use a setter.

Example:

```
@Component
class B {

    private A a;

    public void setA(A a) {
        this.a = a;
    }
}
```

Now `B` can be created without requiring `A` in its constructor.

53. Using `@PostConstruct` to Set the Dependency

Suppose `A` has a reference to `B`.

After `A` is created and dependencies are available, `@PostConstruct` can call the setter on `B`.

Conceptually:

```
@PostConstruct
public void setB() {
    b.setA(this);
}
```

Flow:

```
Create A
↓
A receives B
↓
B already created without A
↓
@PostConstruct on A
↓
B.setA(this)
↓
A → B
B → A
```

The circular dependency is established after object creation instead of during constructor creation.

54. Important Warning About Circular Dependency

The chapter's main recommendation is:

Do not design your application with circular dependencies.

Techniques such as:

```
@Lazy
Setter Injection
@PostConstruct
```

can technically break the creation cycle, but they are workarounds.

Preferred approach:

```
Refactor the design
↓
Remove Circular Dependency
```

55. Final Lifecycle Summary

1. IoC Container starts
↓
2. Configuration is read
↓
3. Components are scanned
↓
4. Bean Definitions are created
↓
5. Bean objects are instantiated
↓
6. Dependencies are injected/resolved
↓
7. Aware callbacks
↓
8. Initialization callbacks
↓
9. Bean Ready
↓
10. Bean Used
↓
11. Destruction callbacks
↓
12. Bean Destroyed

56. Initialization vs Destruction

INITIALIZATION

Bean Created

↓

Dependencies Ready

↓

@PostConstruct

↓

Bean Ready

DESTRUCTION

```
Bean Used
  ↓
Container Closing
  ↓
@PreDestroy
  ↓
Bean Destroyed
```

57. Interview Quick Questions

Q1. What is Bean Lifecycle?

The sequence of stages through which a Spring-managed Bean passes, from creation and dependency injection through initialization, usage, and eventual destruction.

Q2. What is the first major step?

```
IoC Container starts
```

Q3. Does Spring directly create all objects immediately?

Not conceptually. It first reads configuration and creates Bean Definitions containing metadata about the Beans.

Q4. What is a Bean Definition?

Metadata/instructions that describe how Spring should manage a Bean.

Q5. What happens after Bean instantiation?

Dependencies are resolved/injected, followed by applicable lifecycle callbacks.

Q6. What is an Aware Interface?

A Spring interface that allows a Bean to receive framework/container-related information.

Examples:

```
BeanNameAware  
ApplicationContextAware
```

Q7. What is a callback?

A method that is invoked by Spring at a particular lifecycle stage rather than being explicitly invoked by application code.

Q8. What is `@PostConstruct` ?

An annotation used to mark a method that Spring invokes during Bean initialization after the Bean has been constructed and dependencies have been processed.

Q9. What are the three initialization mechanisms discussed?

```
InitializingBean  
init-method  
@PostConstruct
```

Q10. What is `InitializingBean`'s callback method?

```
afterPropertiesSet()
```

Q11. What is `@PreDestroy` ?

An annotation used to mark a method for cleanup before Spring destroys an applicable managed Bean.

Q12. What are the three destruction mechanisms?

```
DisposableBean  
destroy-method  
@PreDestroy
```

Q13. What is `DisposableBean`'s callback method?

```
destroy()
```

Q14. Why use `@PostConstruct` instead of constructor for some initialization?

Because it runs later in the lifecycle, after dependency injection, so initialization logic can use injected dependencies.

Q15. Does Spring manage Prototype Bean destruction?

No. Spring creates and initializes the Prototype instance but does not manage its complete destruction lifecycle.

Q16. What happens to an unused Prototype object?

Once it is no longer referenced, it becomes eligible for normal Java Garbage Collection.

Q17. Does `context.close()` destroy Prototype instances?

No. Spring does not manage Prototype instances' destruction in the same way it manages Singleton Beans.

Q18. Can `@PostConstruct` help break a circular dependency?

Technically, yes, when the dependency design is changed so that one side can be created first and the other reference is established later.

However, the preferred solution is to remove the circular dependency through refactoring.

58. Most Important Things to Remember

```
Bean Lifecycle
→ Creation
→ Dependency Injection
→ Aware callbacks
→ Initialization
```

- Usage
- Destruction

Initialization:
InitializingBean
init-method
@PostConstruct

Destruction:
DisposableBean
destroy-method
@PreDestroy

@PostConstruct
→ initialization callback

@PreDestroy
→ destruction callback

Singleton
→ Spring manages creation + lifecycle + destruction
for the managed Singleton instance

Prototype
→ Spring creates + initializes
→ gives instance to client
→ does not manage its destruction

Aware Interfaces
→ framework/container information
→ specific use cases
→ Spring invokes the callbacks

Circular Dependency
→ Prefer removing/refactoring it
→ @Lazy / setter / @PostConstruct can technically break
the creation cycle

59. One-Line Lifecycle Formula

```
START
→ CONFIGURATION
→ BEAN DEFINITION
→ INSTANTIATION
→ DEPENDENCY INJECTION
→ AWARE CALLBACKS
→ INITIALIZATION CALLBACKS
→ READY
→ USE
→ DESTRUCTION CALLBACKS
→ DESTROY
```

Golden Rule

A Spring Bean is not simply "created and used". Spring manages a complete lifecycle around the Bean — from Bean Definition and instantiation to dependency injection, callbacks, initialization, usage, and destruction.